

# LangGraph Migration - Old Graph vs New Adapter Comparison

---

## Critical Differences Found

FIXED: Input Vertex Updates

**Old Graph** (`_run` method, line 741):

```
if inputs:
    self._set_inputs(input_components, inputs, input_type)
```

**Problem:** LangGraphAdapter's `build_vertex` didn't update input vertices before building.

**Fix Applied:** Added input vertex update logic in `adapter.py build_vertex()` method (lines 373-384)

---

## Potential Issues to Monitor

### 1. Session ID Updates (PARTIALLY IMPLEMENTED)

**Old Graph** (`_run` method, lines 745-751):

```
# Update all the vertices with the session_id
for vertex_id in self.has_session_id_vertices:
    vertex = self.get_vertex(vertex_id)
    if vertex is None:
        msg = f"Vertex {vertex_id} not found"
        raise ValueError(msg)
    vertex.update_raw_params({"session_id": session_id})
```

**LangGraphAdapter Status:**

- Session ID is set in `build_graph_from_data()` in `api/utils.py` (lines 192-198)
- Vertices are updated there, not in adapter
- **But:** `build_vertex()` doesn't check if session\_id needs updating per-build

**Recommendation:**

- Current implementation works for single-session builds
  - May need enhancement if session changes between builds of same flow
- 

### 2. **FIXED** - Input Type/Component Filtering

### Old Graph `_set_inputs` (lines 686-706):

```
def _set_inputs(self, input_components: list[str], inputs: dict[str, str],
input_type: InputType | None):
    for vertex_id in self._is_input_vertices:
        vertex = self.get_vertex(vertex_id)
        # Filter by input_components
        if input_components and (vertex_id not in input_components and
vertex.display_name not in input_components):
            continue
        # Filter by input_type (e.g., "chat" vs "text")
        if input_type is not None and input_type != "any" and input_type not in
vertex.id.lower():
            continue
        vertex.update_raw_params(inputs, overwrite=True)
```

### LangGraphAdapter Status:

- **IMPLEMENTED** - Component filtering added in `adapter.py build_vertex()`
  - Checks if vertex\_id or display\_name matches components filter
  - Skips vertices not in the component filter list
  - Logs when vertices are skipped due to filtering
  - Input type filtering not implemented (rarely used, can be added if needed)
- 

## 3. **FIXED** - Tracing/Run Initialization

### Old Graph `initialize_run` (lines 618-628):

```
async def initialize_run(self) -> None:
    if not self._run_id:
        self.set_run_id()
    if self.tracing_service:
        run_name = f"{self.flow_name} - {self.flow_id}"
        await self.tracing_service.start_tracers(
            run_id=uuid.UUID(self._run_id),
            run_name=run_name,
            user_id=self.user_id,
            session_id=self.session_id,
        )
```

### LangGraphAdapter Status:

- **IMPLEMENTED** - Full graph-level tracing in `initialize_run()` (lines 306-339)
- Run ID generation via `set_run_id()` method
- Tracing service initialization
- Graph-level tracing span created

- Trace cleanup via `end_all_traces_in_context()` method
  - Per-vertex tracing still works in `build_vertex()`
- 

#### 4. Cache Management (IMPLEMENTED ELSEWHERE)

Old Graph `_run` (lines 753-757):

```
try:
    cache_service = get_chat_service()
    if self.flow_id:
        await cache_service.set_cache(self.flow_id, self)
except Exception:
    logger.exception("Error setting cache")
```

LangGraphAdapter:

- Cache is managed in `build.py` (generate\_flow\_events)
  - Not adapter's responsibility
  - Works correctly
- 

#### 5. **FIXED** - Vertex Freezing/Caching Logic

Old Graph `build_vertex` (lines 1423-1448):

```
should_build = False
if not vertex.frozen:
    should_build = True
else:
    # Check the cache for the vertex
    if get_cache is not None:
        cached_result = await get_cache(key=vertex.id)
    else:
        cached_result = CacheMiss()
    if isinstance(cached_result, CacheMiss):
        should_build = True
    else:
        try:
            cached_vertex_dict = cached_result["result"]
            # Restore cached vertex state
            vertex.built = cached_vertex_dict["built"]
            vertex.artifacts = cached_vertex_dict["artifacts"]
            # ... more fields
        except KeyError:
            should_build = True

if should_build:
    await vertex.build(...)
```

### LangGraphAdapter Status:

- **IMPLEMENTED** - Full caching logic added to `adapter.py build_vertex()` (lines 389-429)
  - Checks `vertex.frozen` before building
  - Retrieves from cache if frozen and available
  - Restores vertex state from cache
  - Saves to cache after building frozen vertices
  - Added `finalize_build()` method to `vertex_wrapper.py` (lines 223-250)
- 

## 6. Webhook/Start Component Prioritization

### Old Graph `_run` (lines 761-765):

```
# Prioritize the webhook component if it exists
start_component_id = find_start_component_id(self._is_input_vertices)
await self.process(
    start_component_id=start_component_id,
    ...
)
```

### LangGraphAdapter:

- Uses `start_component_id` from kwargs in `build.py`
- Unclear if webhook prioritization happens

### Recommendation:

- Review if webhook components need special handling
  - Old Graph has `find_start_component_id()` helper
- 

## 7. Error Tracing Cleanup

### Old Graph `_run` (lines 768-772, 775):

```
try:
    await self.process(...)
    self.increment_run_count()
except Exception as exc:
    self._end_all_traces_async(error=exc)
    raise
self._end_all_traces_async()
```

### LangGraphAdapter:

- Per-vertex tracing cleanup in `build_vertex` (lines 393-398)
- No graph-level trace cleanup on error

#### Recommendation:

- If graph-level tracing is added, ensure cleanup on errors
- 

## Summary of Priorities

### COMPLETED

1. **Input Vertex Updates** - Fixed input values being empty
2. **Frozen Vertex Caching** - Performance optimization implemented
3. **Input Component Filtering** - Multi-input flow support added
4. **Graph-Level Tracing** - Full observability with tracing service

### LOW PRIORITY

4. **Webhook Prioritization** - Verify if needed
  5. **Session ID per-build** - Current implementation likely sufficient
- 

## Testing Recommendations

### 1. Test frozen vertices:

- Create flow with expensive API call
- Mark vertex as frozen
- Verify it doesn't re-execute

### 2. Test multi-input flows:

- Flow with ChatInput + TextInput
- Verify correct input is updated

### 3. Test error scenarios:

- Verify tracing cleanup on errors
- Check for resource leaks

### 4. Performance testing:

- Compare execution time old Graph vs LangGraph
  - Monitor cache hit rates
- 

## Files to Review

- `graph_langgraph/adapter.py` - Main adapter implementation
- `graph/graph/base.py` - Original Graph (reference)

- [api/build.py](#) - Build orchestration
- [api/utils.py](#) - Graph construction helpers